> *可以在 $O(\log n)$ 内算出形如 $\sum_{i=1}^{n} f_i x^i$ 这个式子的结果，常见如等比数列，等差比数列等。*

一般还要加上快速幂等[Q]技巧，所以应该是 $O(\log^2 n)$

非质数模数的等比数列求和

考虑最简单的形式：等比数列求和 $\sum_{i=1}^{n} x^i$，已知有简单的公式：

$$\sum_{i=1}^{n} x^i = \frac{x^{n+1}-1}{x-1}$$

可以在 $O(\log n)$ 内求出，但是需要求 $x-1$ 的逆元，如果模数不是质数，那不一定会有逆元，所以不可以直接用求和公式，所以考虑分治求和：
设 $S(n) = \sum_{i=1}^{n} x^i$
这里分 n 的奇偶讨论：
当 n 为奇数时，$S(n) = x^n + S(n-1)$
当 n 为偶数时，$S(n) = S(n/2) + \sum_{i=n/2+1}^{n} x^i$
$= S(n/2) + x^{n/2} \sum_{i=1}^{n/2} x^i$
$= (1 + x^{n/2}) S(n/2)$

直接写个递归就行
从而可以实现 $O(\log^2 n)$ 求出结果

给定三个参数 n, k, a，求：

$$\sum_{i=1}^{n} i^k a^i$$

对 $10^9 + 7$ 取模的结果.
其中 $n \leq 10^{18}, k \leq 2 \times 10^3, a \leq 10^9$
设这个式子是 $S_k(n)$
也是分 n 的奇偶讨论：

> **n 为奇数时：**

$$S_k(n) = a + \sum_{i=2}^{n} i^k a^i$$

$$= a + a \sum_{i=1}^{n-1} (i+1)^k a^i$$

$$= a + a \sum_{i=1}^{n-1} a^i \sum_{j=0}^{k} i^j \binom{k}{j}$$

$\binom{k}{j}$ 是组合数

然后交换求和顺序得：

$$= a + a\sum_{j=0}^{k}\binom{k}{j}\sum_{i=1}^{n-1}a^i i^j$$

$$= a + a\sum_{j=0}^{k}\binom{k}{j}S_j(n-1)$$

> **n 为偶数时：**

$$S_k(n) = \sum_{i=1}^{n/2} i^k a^i + \sum_{i=n/2+1}^{n} i^k a^i$$

$$= S_k(n/2) + a^{n/2}\sum_{i=1}^{n/2}(i+n/2)^k a^i$$

然后和前面的差不多，二项式展开然后交换求和顺序，最后得：

$$= S_k(n/2) + a^{n/2}\sum_{j=0}^{k}\binom{k}{j}(n/2)^{k-j}S_j(n/2)$$

然后做一下记忆化，就能做到在 $O(k^2 \log n)$ 内实现

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

const int mod = 1e9 + 7;
const int maxn = 2e3 + 10;
int n, a, k;
int c[maxn][maxn];

void init()
{
    for(int i = 0; i <= k; i++)
    {
```

```
            c[i][0] = c[i][i] = 1;
            for(int j = 1; j < i; j++)
                c[i][j] = (c[i - 1][j - 1] + c[i - 1][j]) % mod;
        }
}

int quickpow(int a, int b)
{
        int res = 1;
        while (b)
        {
            if (b & 1) res = (res * a) % mod;
            a = (a * a) % mod;
            b >>= 1;
        }
        return res;
}

// 考虑记忆化这个 k 来优化
vector<int> solve(int n)
{
        if (n == 1)
            return vector<int>(k + 1, a);
        vector<int> S(k + 1);
        if (n & 1)
        {
            auto T = solve(n - 1);
            for (int i = 0; i <= k; i++)
            {
                int res = 0;
                for (int j = 0; j <= i; j++)
                    res = (res + (c[i][j] * T[j]) % mod) % mod;
                S[i] = (a + ((a * res) % mod)) % mod;
            }
        }
        else
        {
            auto T = solve(n >> 1);
            int base = quickpow(a, n >> 1);
            for(int i = 0; i <= k; i++)
            {
                int res = 0, p = 1;
                for(int j = i; j >= 0; j--, p = (p * ((n >> 1) % mod)) % mod)
                    res = (res + (((c[i][j] * p) % mod * T[j]) % mod)) % mod;
```

```
                S[i] = (T[i] + (base * res) % mod) % mod;
            }
        }
        return S;
}

signed main()
{
        cin >> n >> a >> k;
        init();
        vector<int>ans = solve(n);
        cout << ans[k];
        return 0;
}
```